Andres Garcia Barros
Github: AmgBar
ID: 921107769
CSC615 Embedded Linux

Assignment 3 – Start your motor

Description:

Using a Raspberry Pi and a motor connected to the Waveshare Motor Driver HAT, create a program that uses i2c communication between the Raspberry Pi and the Motor Driver HAT to make the motor run forward at 100% speed for two seconds, then gradually slow down to 15% speed. The motor will stop for one second before gradually speeding to 100% in the other direction. The program will run, but the motor behavior will not initiate until a button is pressed, which will be connected via a breadboard.

Approach / What I Did:

This assignment involved using I2C communication between the Raspberry Pi and the waveshare Motor Driver Hat. I learned a little about I2C communication during class lectures, but have never worked hands-on with it until this assignment.

Since the motor driver hat connects directly on top of the Raspberry Pi, I simply connected it and wired the motor to the motor driver hat.

To begin I installed i2c-tools to play around with the motor using i2c communication before using C code to control the motor. I used i2cdetect to see the motor driver hat in the Raspberry Pi's i2c bus.

```
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $ i2cdetect -y 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: 40 -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: 70 -- -- -- -- -- -- --
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $
```

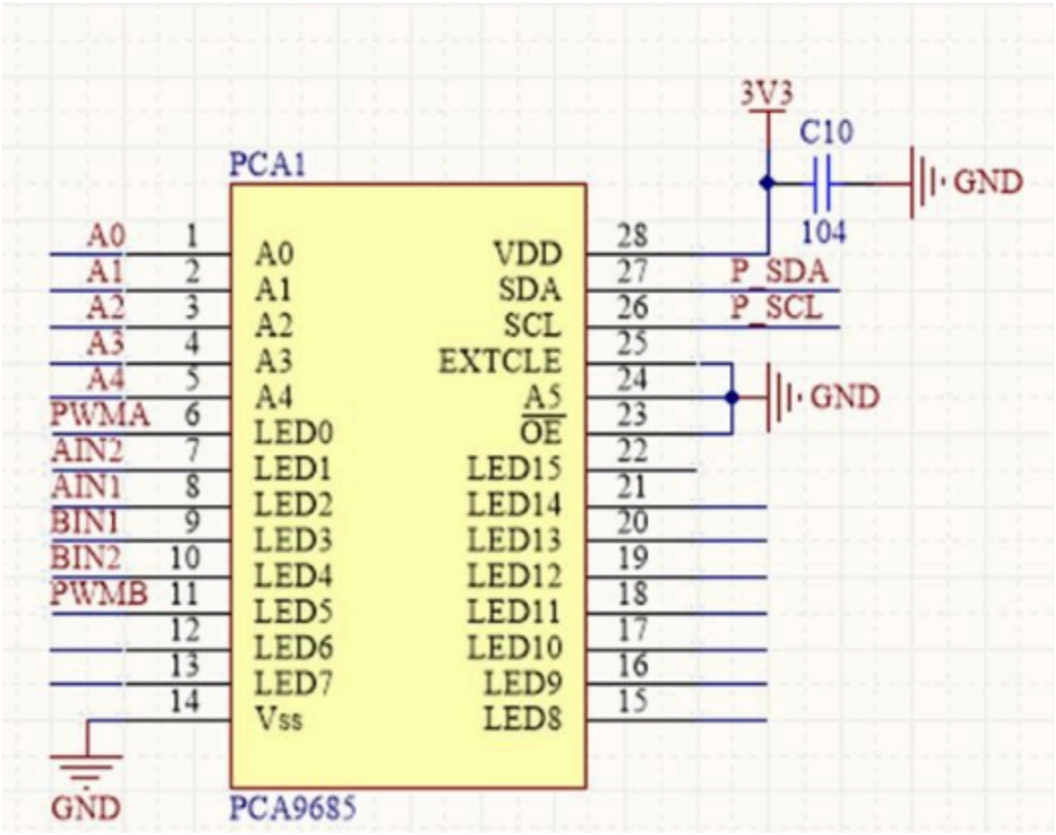
Here it shows that there are two devices, but upon further investigation, I learned that the two addresses refer to the same device

The documentation for the motor driver hat states that it uses a PCA9685 chip to control the motor functions and that PCA9685 is the device that is displayed as connected to the i2c bus.

Using i2cdump I was able to see the registers of the PCA9685 chip. These registers are what is to be modified in order to control the motors connected to the motor driver HAT.

```
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $ i2cdump -y 1 0x40
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 11 04 e2 e4 e8 e0 00 00 00 10 00 00 00 10 00 00    ??????...?...?..
10: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
20: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
30: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX    .?...?XXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX 00 00 00 00 1e 00    XXXXXXXXXX....?.
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $
```

According to the Waveshare documentation on the Motor Driver Hat, only registers which control LED0 to LED5 are relevant to motor control.



Reading the PCA9685 chip datasheet, I found that the registers associated with pins LED0 to LED5 are registers 0x06 to 0x1D

In order to change the values in those registers, I used i2cset command to manipulate them from the command line and play around with the motor. After playing with the motor this way, I had a pretty good understanding of how the motor driver hat communicates with the Raspberry Pi through i2c.

After this I felt ready to use C code to make the program that was assigned. I decided to use pigpio since it is the library which I have used for all the assignments so far and it also has dedicated functions for i2c communication. After understanding how to use I2C to make changes to registers and what their values mean, it was relatively simple to translate that to C code. I first defined all the relevant register addresses in the PCA9685 chip that controlled motor functions, then created specific methods for motor functionality. I created a method for setting the registers to their default values to use once the program is finished, a method for setting the motor speed to a specified whole percentage, and a method to control the on or off state of an LED/motor input.

After that, it was straightforward to make the motor behave as specified in the assignment description. I simply programmed the motor behavior, then used gpio functions to wait for an input, which will be given once a button is pressed, which will exit out of a while loop in the program before starting the motor.

Issues and Resolutions:

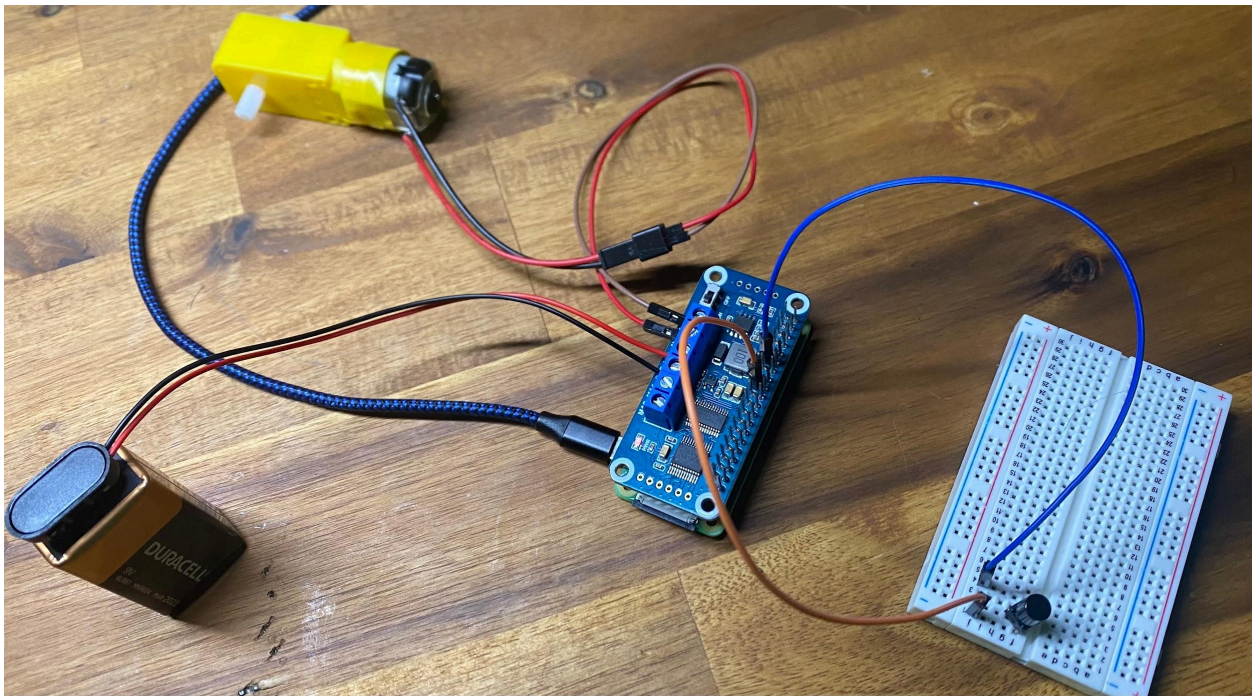
One of the drawbacks of using i2ctools to control the motor is that you can change one register at a time. This made hard to figure out how exactly the speed was controlled through the registers. I understood that the 4th most significant bit of the LED5_ON_H register controlled whether the motor would be 100% or 0% but I wasn't sure how to get the motor to specific fractional speeds. Once I started using C to control the motor I got a better understanding of what values correlate with speed after using a for loop to see the changes. Since the motor driver hat uses 12 bits to control speed that means a value of 4095 is the max you can use to make a value with speed. I realized that speed was inversely proportional to the values so 4095 was essentially off while 1 was equal to max speed.

Having figured out how the values correlate with the motor's speed I now had to figure out how to move those values into the registers since the 8 least significant bits of the value had to go into the LED5_ON_L register and the 4 most significant bits of the speed value had to go into least significant half of the LED5_ON_H register. I solved this by taking the value which represented speed and used bitwise operations to separate the values and put it into their proper registers.

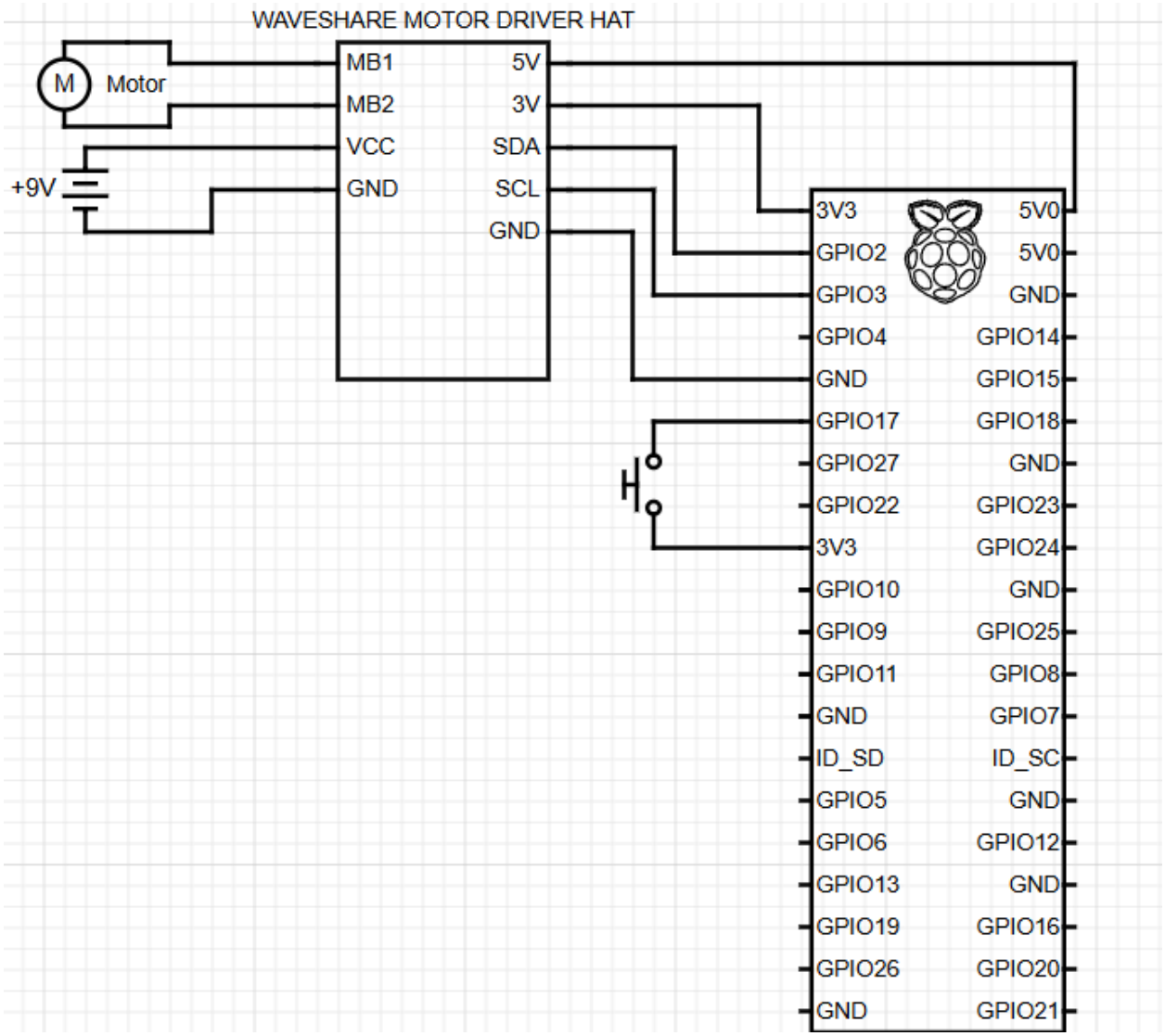
```
    float truePercentage = (100 - percent) / 100.0;
    int calculatedSpeedValue = (MAX_SPEED_VALUE * truePercentage) / 1;
    //Separate speed value's bits in order to put them in their respective registers
    int valueInLow = calculatedSpeedValue & 0xFF;
    int valueInHigh = (calculatedSpeedValue >> 8) & 0xF;

    i2cWriteByteData(motorDriverHandle, LED5_ON_L, valueInLow);
    i2cWriteByteData(motorDriverHandle, LED5_ON_H, valueInHigh);
```

Photo of completed circuit:



Hardware Diagram:

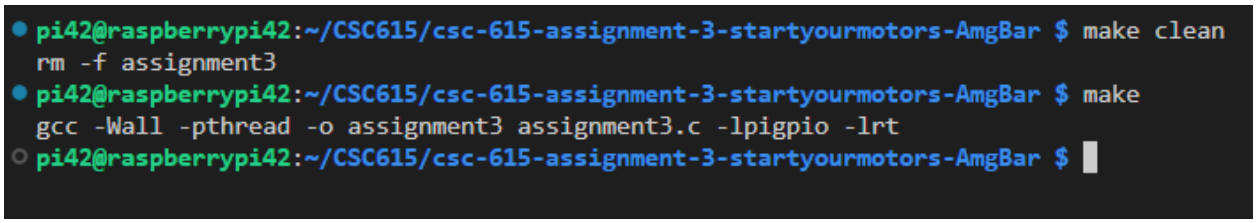


The Waveshare Motor Driver Hat will ideally just sit on top of the Raspberry Pi, but these are the relevant connections it has to the Raspberry Pi

Analysis:

(If required for the assignment)

Screen shot of compilation:



Screen shot(s) of the execution of the program:

```
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $ make clean
rm -f assignment3
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $ make
gcc -Wall -pthread -o assignment3 assignment3.c -lpigpio -lrt
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $ make run
sudo ./assignment3
Press button to start motor
SETTING TO 100% SPEED
MAX SPEED FOR 2 SECONDS
SLOWING TO 15 PERCENT
SETTING TO 100% SPEED
SETTING TO 95% SPEED
SETTING TO 90% SPEED
SETTING TO 85% SPEED
SETTING TO 80% SPEED
SETTING TO 75% SPEED
SETTING TO 70% SPEED
SETTING TO 65% SPEED
SETTING TO 60% SPEED
SETTING TO 55% SPEED
SETTING TO 50% SPEED
SETTING TO 45% SPEED
SETTING TO 40% SPEED
SETTING TO 35% SPEED
SETTING TO 30% SPEED
SETTING TO 25% SPEED
SETTING TO 20% SPEED
SETTING TO 15% SPEED
STOPPING FOR 1 SECOND
SETTING TO 0% SPEED
REVERSING
SETTING TO 0% SPEED
SETTING TO 5% SPEED
SETTING TO 10% SPEED
SETTING TO 15% SPEED
SETTING TO 20% SPEED
SETTING TO 25% SPEED
SETTING TO 30% SPEED
SETTING TO 35% SPEED
SETTING TO 40% SPEED
SETTING TO 45% SPEED
SETTING TO 50% SPEED
SETTING TO 55% SPEED
SETTING TO 60% SPEED
SETTING TO 65% SPEED
SETTING TO 70% SPEED
SETTING TO 75% SPEED
SETTING TO 80% SPEED
SETTING TO 85% SPEED
SETTING TO 90% SPEED
SETTING TO 95% SPEED
SETTING TO 100% SPEED
pi42@raspberrypi42:~/CSC615/csc-615-assignment-3-startyourmotors-AmgBar $
```